CONCURRENCY II

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

Administration

- Lab2 – Due NEXT Friday: (2/27)
  - I **Strongly** recommend submitting **TOMORROW 2/20 (end of week 2/3)**
    - Both parts of the lab
  - It will take **multiple attempts** and **strong unit testing** to pass all autograder tests this time.
  - Remember, we have piazza support and many office hours. If you're feeling stuck or lost, please reach out!
    - The most common failure scenario is procrastination, driven by hesitation

Lab 2	JAN 31, 2025 8:00 AM	FEB 21, 2025 11:59 PM Late Due Date: FEB 24, 2025 11:59 PM	23

- Lab3 – coming soon to the lab near you

System calls, interrupts, and exceptions

Interrupts are a pseudo concurrency!

When we program in an interruptible environment, we need to think about safety in concurrency.

Concurrency considerations

Kernel code

Lock()

Shared = 7

unlock()

Int handler

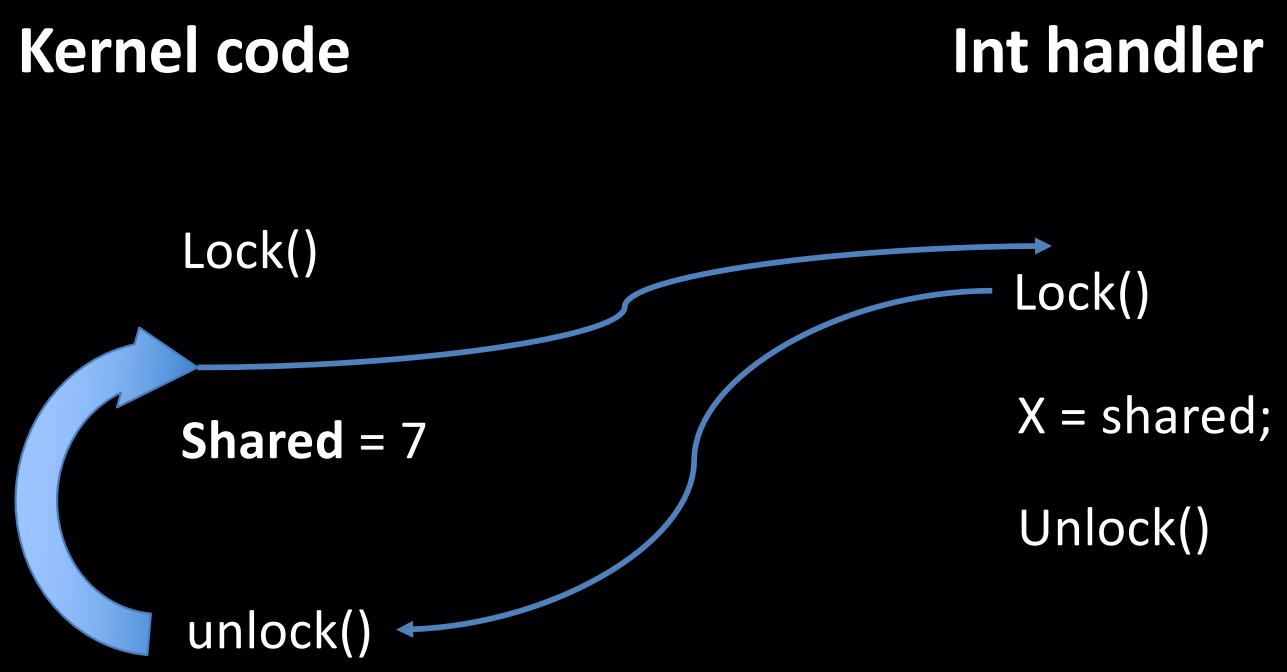
Lock()

X = shared;

Unlock()

- Can't make interrupt handler ignore the locks
- Delay the interrupt?
- Whenever I'm holding a lock, delay the interrupts

Concurrency considerations



- Can't make interrupt handler ignore the locks
- Delay the interrupt?
- Whenever I'm holding a lock, delay the interrupts

Concurrency considerations

Kernel code

Interrupt disable()

Lock()

Shared = 7

unlock()

Interrupt enable()

Int handler

Lock()

X = shared;

Unlock()

- Can't make interrupt handler ignore the locks
- **Delay the interrupt?**
- Whenever I'm holding a lock, delay the interrupts

Concurrency considerations

Kernel code

Interrupt disable()

Lock()

Shared = 7

unlock()

Interrupt enable()

order

Int handler

Lock()

X = shared;

Unlock()

- Can't make interrupt handler ignore the locks
- **Delay the interrupt?**
- Whenever I'm holding a lock, delay the interrupts

Concurrency considerations

Georgia Tech

Kernel code

Acquire()

Interrupt disable()

Lock()

Shared = 7

Release()

unlock()

Interrupt enable()

Int handler

Lock()

X = shared;

Unlock()

- Can't make interrupt handler ignore the locks
- **Delay the interrupt?**
- Whenever I'm holding a lock, delay the interrupts

System calls, interrupts, and exceptions

When we program in an interruptible environment, we need to think about safety in concurrency.

In xv6 we avoid interrupt deadlock by **disabling interrupts** when a **spinlock** is held!

Mutual Exclusion with an interrupt handler is achieved with **interrupt disable**

Too Much Milk?

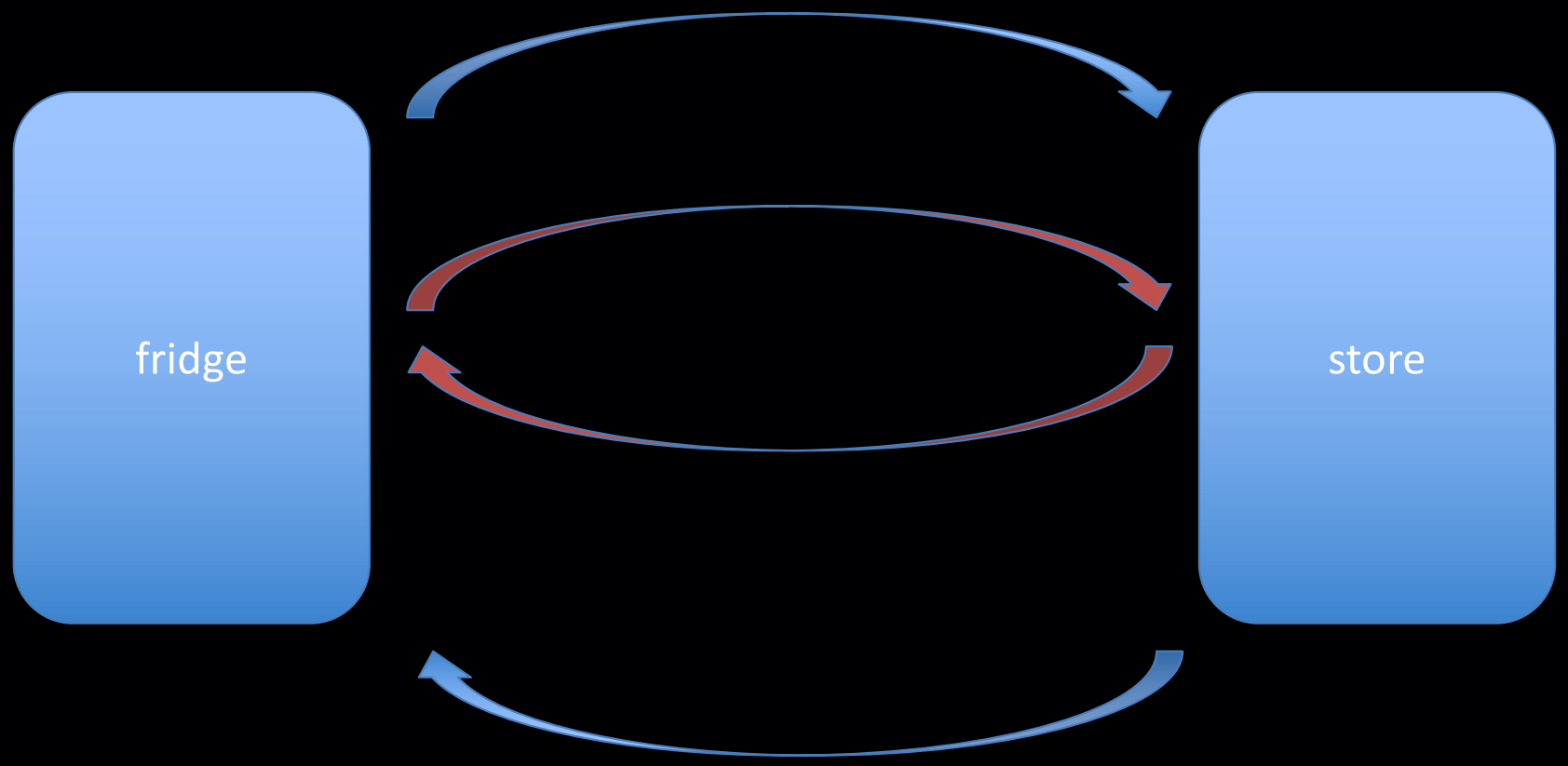
How familiar is everyone with thread communication and locking?

- Have you covered the basics how to construct locks?
- Have you tried to build locks with only atomic store/load?

Classic Motivating Example

- Difficulty of providing mutual exclusion with concurrent memory reads/writes
  - Jumping ahead : need hardware support
- Universal correctness must be assured
  - Code must work ALL the time, not just MOST of the times
  - Concurrency bugs occur only for SOME of the execution interleavings
- Mutual exclusion design: mental model for concurrent updates
  - Mindful of what is *checked* and what is *updated*
  - *Mindful of shared data struct access BETWEEN check and update*

Too Much Milk?



Thread Coordination

Bob

- Check fridge -> no milk
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Alice

- Check fridge -> no milk
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Thread Coordination: Problematic Interleaving

Bob

- Check fridge -> no milk
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Alice

- Check fridge -> no milk
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Formalizing "Too Much Milk"

- Shared variables
  - "check fridge" → read a shared variable
  - "put milk in fridge" → update a shared variable

- *Safety* property
  - At most one person buys milk
- *Liveness*:
  - Someone buys milk when needed

- How can we solve this problem??

Thread Coordination: Naive

Bob

- Check fridge -> no milk
- Leave a note
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Alice

- Check fridge -> no milk
- Leave a note
- Go to store
- Buy milk
- Arrive home; milk -> fridge

Liveness?

Safety?

MULTIPROCESSORS AND LOCKING

- Can we build a lock with only atomic stores and atomic loads?
  - Peterson's Algorithm.

```
P0:       flag[0] = true;
P0_gate:  turn = 1;
          while (flag[1] == true && turn ==
1)
          {
              // busy wait
          }
          // critical section
          ...
          // end of critical section
          flag[0] = false;
```

```
P1:       flag[1] = true;
P1_gate:  turn = 0;
          while (flag[0] == true && turn ==
0)
          {
              // busy wait
          }
          // critical section
          ...
          // end of critical section
          flag[1] = false;
```

Source: https://en.wikipedia.org/wiki/Peterson%27s_algorithm

MULTIPROCESSORS AND LOCKING

- Can we build a lock with only atomic stores and atomic loads?
  - Peterson's Algorithm.
  - Completely Impractical!
  - We need a better concurrency mechanism
    - Atomic read and write in one operation

MULTIPROCESSORS AND LOCKING

- c11 provides several atomic operations:
  - https://en.cppreference.com/w/c/atomic
  - atomic_load()
  - atomic_store()
  - atomic_fetch_add()
  - atomic_exchange() ← An atomic load and store together!
  - atomic_compare_exchange() ← Even fancier (cond. + load + store)

LOCKING

- Can we solve too much milk with c11 ***atomic exchange***?

```
// atomic swap (xchg)
Exchange(int *y, x) {
    t = *y;
    *y = x;
  return t;
}
```

```
// lock == 1 is locked

r = 1
while (r != 0) {
  r = exchange(&lock,1)
}
```

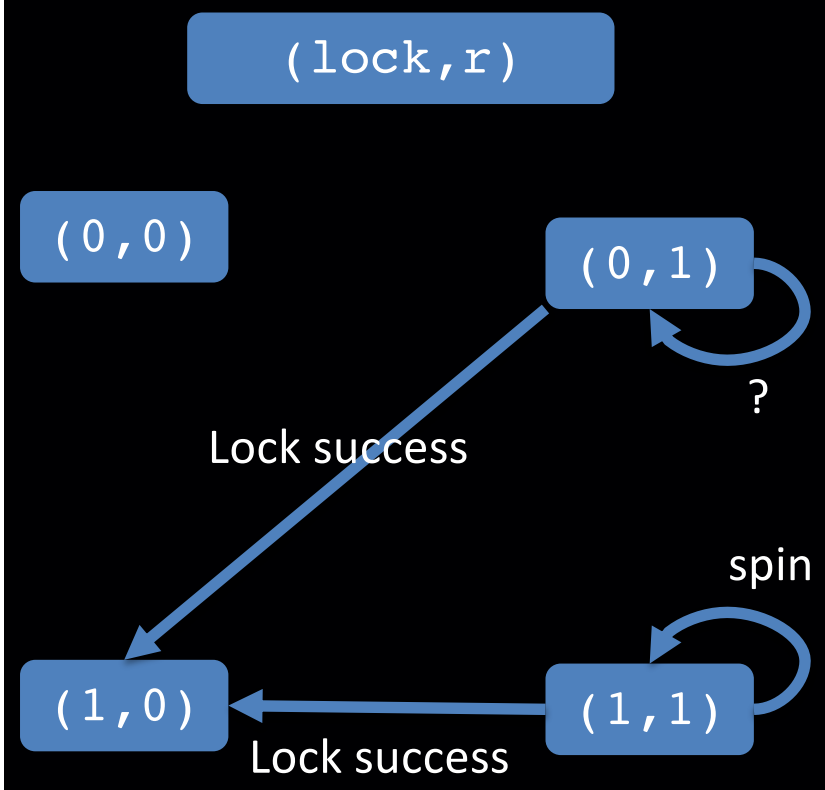
```
// e.g. initial cond: lock==0
// e.g. initial cond: lock==1
```

Spin until you lock

spin lock

LOCKING

- Can we solve too much milk with c11 atomic exchange?




```
// lock == 1 is locked

r = 1
while (r != 0) {
  r = exchange(&lock,1)
}
```

```
// Example: lock==0

// Example: lock==1
```

Georgia Tech

What value pairs can %eax and %ebx with?

Initial: x = y = 0;

T1	T2
movl $1, (x)	movl $1, (y)
movl (y), %eax	movl (x), %ebx

What values can %eax and %ebx end with?

A. %eax = 0, %ebx = 0

B. %eax = 0, %ebx = 1

C. %eax = 1, %ebx = 0

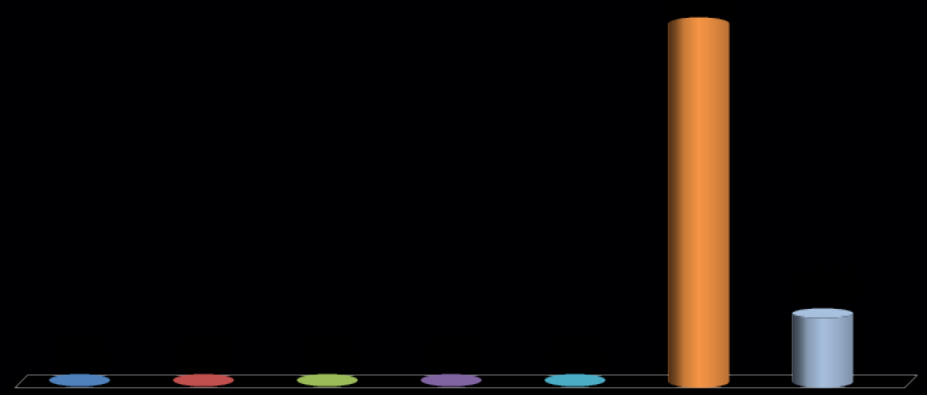
D. %eax = 1, %ebx = 1

E. B, C

F. B, C, D

G. A, B, C, D

T1	T2
movl $1, (x) movl (y), %eax	movl $1, (y) movl (x), %ebx



What values can %eax and %ebx end with?

A. %eax = 0, %ebx = 0

B. %eax = 0, %ebx = 1

C. %eax = 1, %ebx = 0

D. %eax = 1, %ebx = 1

T1

```
movl $1, (x)
movl (y), %eax
```

T2

```
movl $1, (y)
movl (x), %ebx
```

B

```
movl $1, (x)
movl (y), %eax

movl $1, (y)
movl (x), %ebx
```

C

```
movl $1, (y)
movl (x), %ebx

movl $1, (x)
movl (y), %eax
```

D

```
movl $1, (y)
movl $1, (x)

movl (x), %ebx
movl (y), %eax
```

A???

```
movl (x), %ebx
movl (y), %eax

movl $1, (y)
movl $1, (x)
```

What values can %eax and %ebx end with?

A. %eax = 0, %ebx = 0

B. %eax = 0, %ebx = 1

C. %eax = 1, %ebx = 0

D. %eax = 1, %ebx = 1

T1	T2
`movl $1, (x)`	movl $1, (y)
`movl (y), %eax`	movl (x), %ebx

A???

```
movl (x), %ebx
movl (y), %eax

movl $1, (y)
movl $1, (x)
```

System calls, interrupts, and exceptions

What value pairs can %eax and %ebx with?

Initial: x = y = 0;

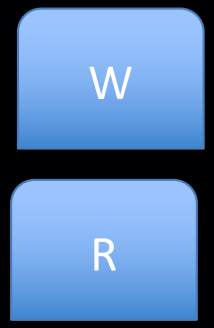
T1	T2
movl $1, (x)	movl $1, (y)
movl (y), %eax	movl (x), %ebx

x86 uses a "memory model" called "total store order" – This allows loads to be re-ordered

TSO Description Space

- Will cover in detail later:
  - Essentially, ***Stores are totally ordered***, but there isn't a guarantee over load orderings – they can be reordered
  - TSO: Respects W-W and R-W dependencies,
  - TSO: but not W-R dependencies

```
movl $1, (y)
movl (x), %ebx
```



Memory Model Summary

Georgia Tech

- We naturally assume "sequential consistency" when programming:
  - Sequential consistency means every statement executes in order
- This is **wrong.** We are not promised sequential consistency by our CPU, our programming language, or our compiler!
  - Instead, x86 promises Total Store Order (TSO) with ambiguous semantics!
  - C/C++ as of version 11 offer "**DRF0**" like semantics – if you don't have a data-race, then your code will be sequentially consistent
    - They also offer some atomic primitives… (although these are dangerous)

Takeaway: DRF0 = no data race => sequential consistency

Writing Concurrent Code

- If I want to write concurrent code, what do I do?
  - Require proper concurrency!
  - Locks, semaphores, condition variables, barriers, in (very) *rare* cases c11 atomics

lab2

- These all rely on special operations, they build up ordering and atomicity abstractions, just like other OS concepts.

Takeaway

- If you write data race free (DRF) code – then you are guaranteed that your code will behave in a sequentially consistent manner
- So make sure you write DRF code!
- Use provided and implemented abstractions for synchronization primitives any time there's a possibility of having a data race
- Definition of a data race: possibility of > 1 access to a shared memory location, with at least one of them being a write.
- Think about your code from first principles, use FSA, check for safety and liveness.

MULTIPROCESSORS AND LOCKING

Data-races: -- ***Two concurrent memory accesses to the same location, at least one of which is a write***

Occur in our kernel (e.g. Multiple processes read/write the same file)

Use a critical section to protect

- Locking primitive
- Acquire and Release

Race conditions are often hard to reproduce and troubleshoot

- Think about: primitives accessed by multiple threads
- How do you protect it with a lock?
- Suggestion: better to overlock then underlock (correctness > perf)

Georgia Tech

Deadlock

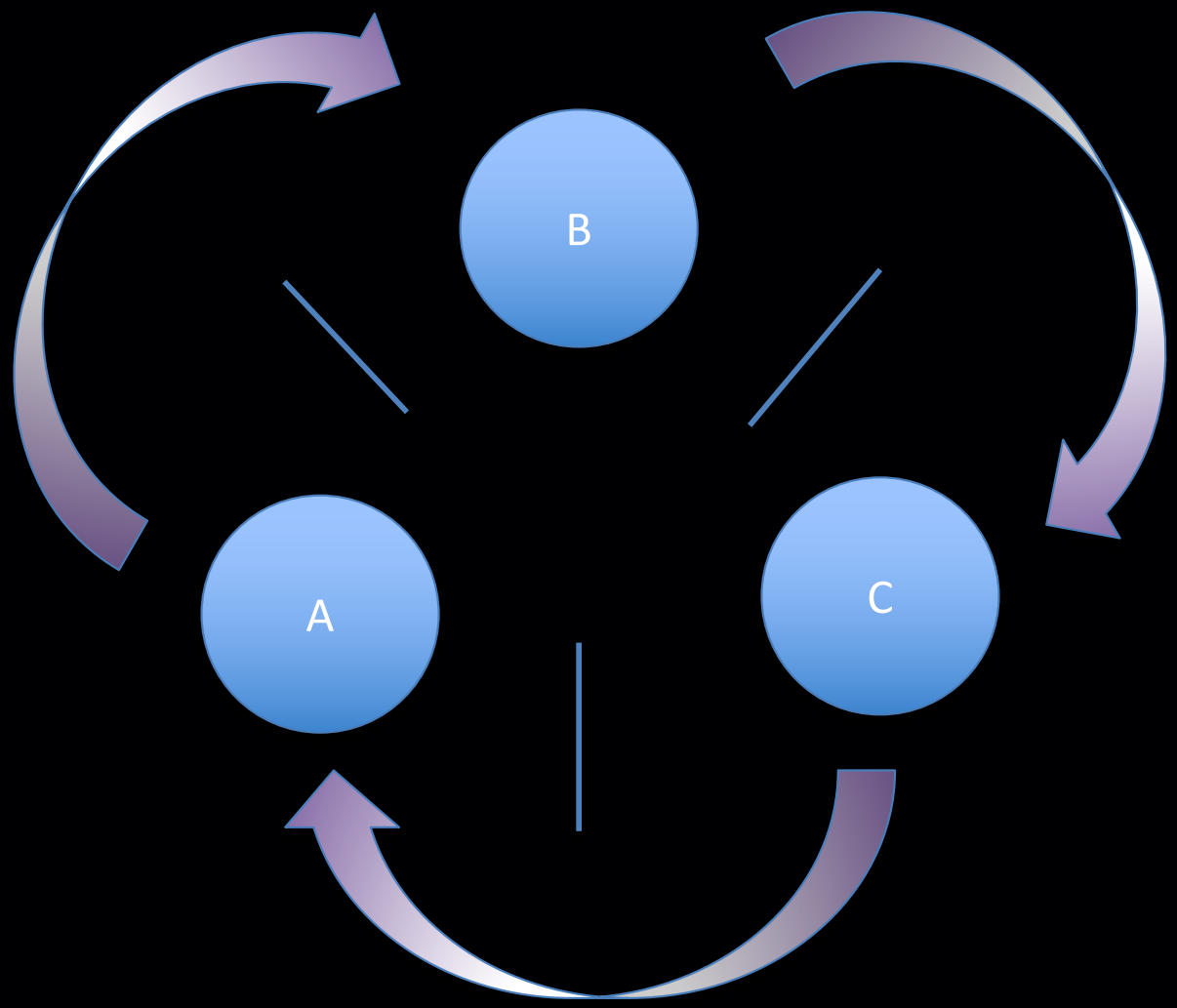
Ex. Dining Philosophers

May need to hold multiple locks to execute a task

xv6 uses a max of two locks. Ex:

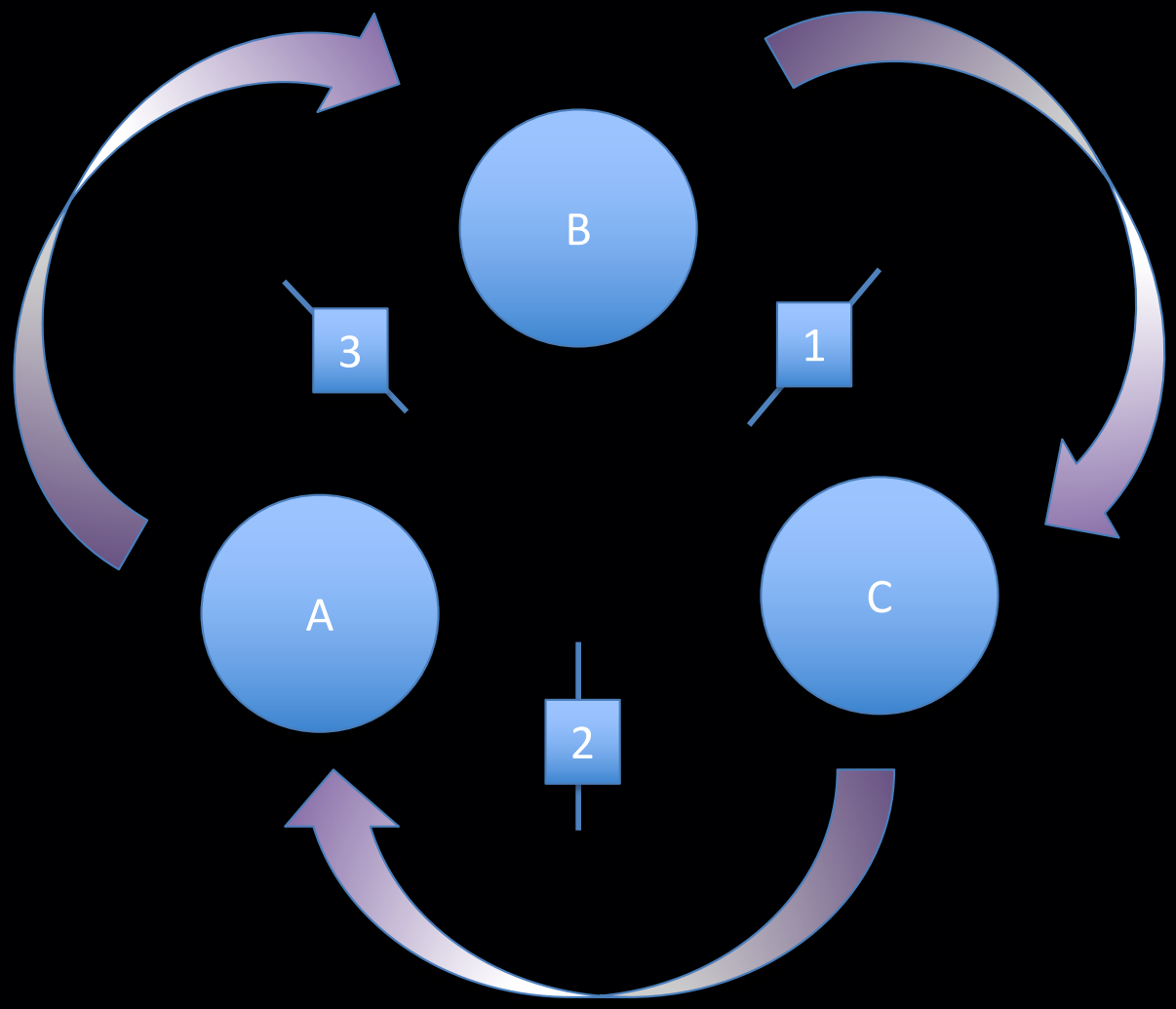
- ideintr holds ide locks, but also calls wakeup which acquires the ptable lock
- More examples in file system (often must lock directory along with file)

DINING PHILOSOPHERS PROBLEM



How can we fix this problem?

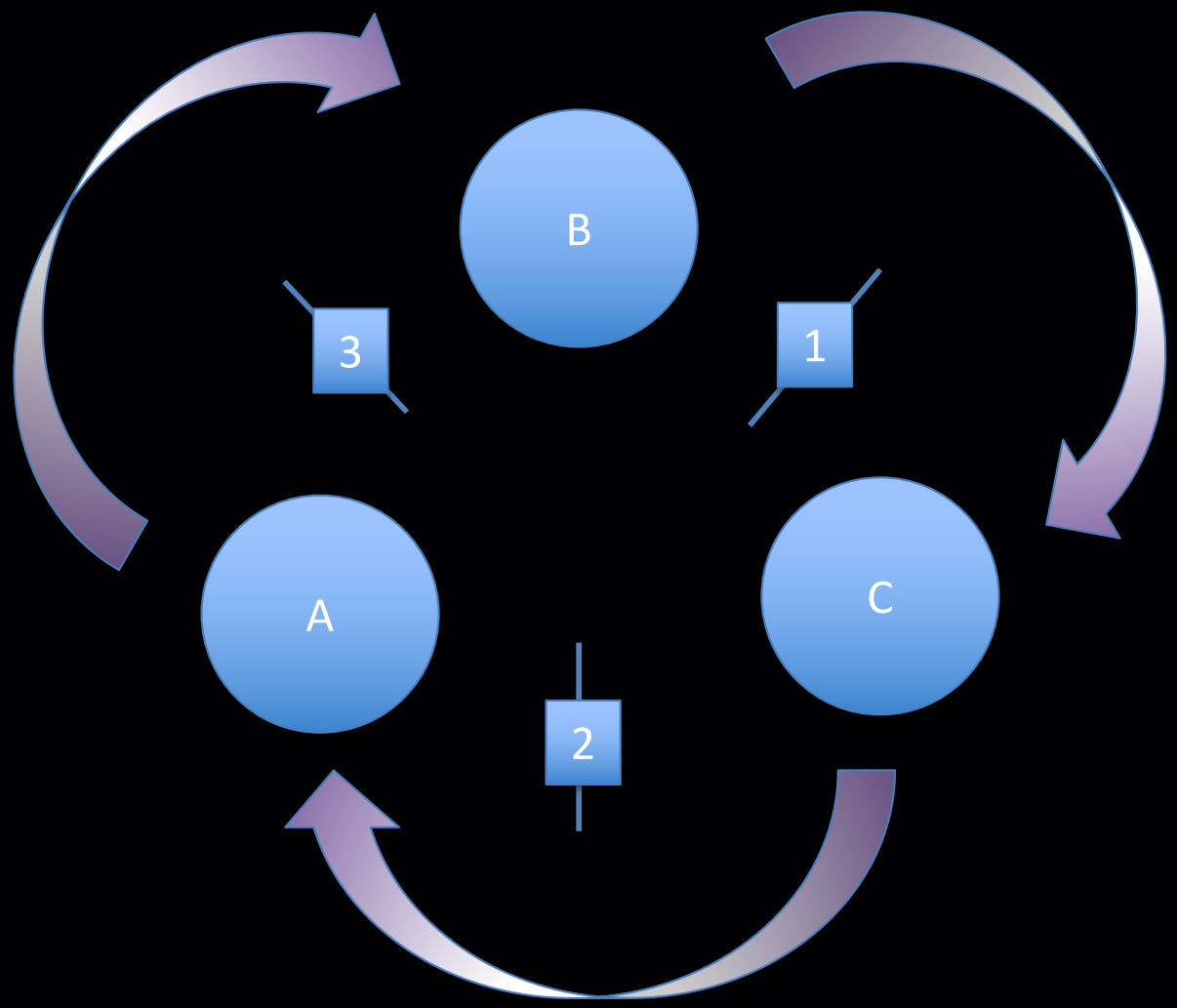
DINING PHILOSOPHERS PROBLEM



Break the cycle of doom

Impose order

DINING PHILOSOPHERS PROBLEM



How was the cycle broken?

Concurrency considerations

Kernel code

Int handler

- Delay the interrupt?
- Whenever I'm holding a lock, delay the interrupts

Impose order

Georgia Tech

Instruction and Memory Reordering

Modern compilers and processors support out of order execution

- Concurrency may expose a hazard due to reordering
- Solution: Tell compiler not to reorder (use c11 atomic primitives)

This is old, use c11 atomics instead (they have language defined behavior)

Georgia Tech

Volatile

- **Misunderstood!**
- A keyword assigned to c/c++ variables to tell compiler that variable may change unexpectedly
  - Not a substitute for atomic operations!
  - Does not enforce memory consistency or fencing!
- When is it useful?
  - For writing memory mapped device drivers – that's about it.
  - https://www.kernel.org/doc/html/latest/process/volatile-considered-harmful.html

Georgia Tech

Volatile

C programmers have often taken volatile to mean that the variable could be changed outside of the current thread of execution; as a result, they are sometimes tempted to use it in kernel code when shared data structures are being used. In other words, they have been known to treat volatile types as a sort of easy atomic variable, which they are not. The use of volatile in kernel code is almost never correct; this document describes why.

- When is it useful?
  - For writing ***memory mapped device drivers*** – that's about it.
  - https://www.kernel.org/doc/html/latest/process/volatile-considered-harmful.html

Georgia Tech

Spinlock shortfalls

Complex groups of functions that may call each other (allocproc, fork, userinit, ptable.lock)

If everyone acquires lock, we have deadlock

Solution: Enforce Locking Rules

- Kernel programmer must have awareness of what locks should be held
- Another solution: recursive locks (complex)
- Other examples include pipe read/write complexity (who holds the lock?)

MULTIPROCESSORS AND LOCKING

Real world

Concurrency and parallel programming are active areas of research

Better to use primitive locks to form higher level constructs

- Abstract away locking
- **xv6** does not do this
- libraries like ***pthreads*** provide higher level locking capability

Can implement atomic locks without hardware support, but expensive and complex

Another option: lock free data structures and algorithms

Many issues w/ performance related to cache lines, multiple processors, etc.

Spinlock in xv6

Pretty much same, but provide debugging info

```
struct spinlock {
  atomic_uint locked;       // Is the lock held?

  // Q?
  char *name;        // Name of lock.
  struct cpu *cpu;   // The cpu holding the lock.
  uint pcs[10];      // The call stack (an array of program counters)
                     // that locked the lock.
};
```

Georgia Tech

acquire() in xv6

```
01  void acquire(struct spinlock *lk) {
02    pushcli(); // disable interrupts to avoid deadlock.
03    if (holding(lk))
04      panic("acquire");
05
06    // atomic operation.
07    // It also serializes, so that reads after acquire are not
08    // reordered before it.
09    while (atomic_exchange_explicit(&lk->locked, 1,
                                      memory_order_acq_rel) != 0)
10      ;
11
12    // Record info about lock acquisition for debugging.
13    lk->cpu = cpu;
14    getcallerpcs(&lk, lk->pcs);
15  }
```

Georgia Tech

release() in xv6

```
01  void release(struct spinlock *lk) {
02    if (!holding(lk))
03      panic("release");
04
05    lk->pcs[0] = 0;
06    lk->cpu = 0;
07
08    // Q?
09    atomic_store_explicit(&lk->locked, 0,
                                memory_order_release);
10
11    popcli();
12  }
```